

Design, Assembly, and Vision-Based Gesture Control of an Open-Source Amazing Hand

1st Mo Chen

The School of The Gifted Young
University of Science and Technology of China
Hefei, China
cm_xian@mail.ustc.edu.cn

2nd Haoxuan Liu

The School of The Gifted Young
University of Science and Technology of China
Hefei, China
lhx090318@mail.ustc.edu.cn

3rd Jinhao Xu

The School of The Gifted Young
University of Science and Technology of China
Hefei, China
xjh_ustc@mail.ustc.edu.cn

4th Yifu Cai

Department of Astronomy
University of Science and Technology of China
Hefei, China
yifucai@ustc.edu.cn

5th ShuoHao Wang

University of Science and Technology of China
Hefei, China
shuohaowang@mail.ustc.edu.cn

Abstract—This report presents the final course project based on the open-source Amazing Hand. The work started from a literature review on robotic hands, grasp types, and common design trends. After that, we studied basic control ideas such as PID control, assembled a real Amazing Hand prototype, tested basic servo motion, and built several forms of gesture control. The hardware work included 3D-printed parts, finger assembly, wiring, base fixing, zero-position tuning, middle-offset tuning with a self-written safe fine-tuning program, and repair of a broken printed part. The software work included a basic Python motion demo, an open-source hand tracking pipeline with MediaPipe, dora-rs, MuJoCo simulation, and real hardware output, a screen-based tracking extension, and an independently written stable hand tracking program. The final program maps MediaPipe hand landmarks to finger curl and splay commands, adds neutral splay calibration, applies temporal filtering and curl locking, and sends smoothed target angles to eight SCS0009 servos. The project shows that a low-cost open-source hand can be used as a practical learning platform for mechanical assembly, visual sensing, and real-time control. It also shows the main limits of our present system, including lack of tactile feedback, sensitivity to visual occlusion, and limited quantitative testing.

Index Terms—robotic hand, Amazing Hand, hand tracking, MediaPipe, gesture control, open-source robot, servo control

I. Background and Objectives

A. Course Background

Robotic hands are important in robotics because the human hand is one of the main tools that people use to work with the world. A human hand can bend, spread, touch, hold, and move objects in many different ways. It has many joints and a very rich sense of touch. For this reason, building a robot hand that is close to a human hand is a hard task. It is not only a mechanical problem. It is also a control, sensing, and software problem.

At the beginning of this course project, our group first read several papers about robotic hands. The review helped us understand that robotic hand design has changed over a long period. Early research often tried to copy the human hand with many actuators and many sensors. This type of design can be very dexterous, but it is often heavy, costly, and hard to control. Later work paid more attention to practical use. Many systems used simpler driving structures, underactuated mechanisms, soft structures, and lower-cost parts. These designs do not fully copy the human hand, but they can be more useful in real tasks because they are easier to build and easier to control.

We also learned about grasp taxonomies. The GRASP taxonomy gives a standard way to describe human grasp types, such as power grasp, precision grasp, and intermediate grasp [2]. This idea is useful for a course project because it reminds us that a robot hand should not only move each joint. It should also show meaningful hand functions. In our project, we did not try to test all grasp types. Instead, we used basic gestures and simple grasp-like motions to check whether our assembled hand and control system worked in a clear and repeatable way.

After the literature review, we studied basic control knowledge, including PID control. PID control was not the main algorithm in our final code, but it helped us understand the relation between command, response speed, error, and stability. This knowledge was useful when we tuned the hand motion. For example, fast motion can make the system look responsive, but too fast or too large motion may also cause shaking, overshoot, or unsafe mechanical stress. This trade-off appeared many times

when we adjusted servo speed, angle range, smoothing, and hand tracking parameters.

B. Why We Chose Amazing Hand

The Amazing Hand is an open-source, low-cost, 3D-printable robotic hand project [8]. It has four fingers and eight degrees of freedom. Each finger is driven by two small Feetech SCS0009 servos through a parallel mechanism. The design puts all actuators inside the hand, so the hand does not need long tendon cables running to a forearm. The project also provides CAD files, assembly guides, basic Python and Arduino examples, and a more advanced demo directory with simulation and hand tracking tools.

This platform matched our course goal well. First, it gave us a complete hardware base, so we could focus on assembly, tuning, and control instead of designing a whole hand from zero. Second, the hand still had enough complexity to be a real engineering task. It had eight servos, many printed parts, tight wiring, zero-position calibration, and possible mechanical errors. Third, it supported both basic motion scripts and more advanced visual control. This made it possible for the project to grow step by step.

In a course project, a good platform should be open enough for modification. Amazing Hand meets this need. We could run the official basic motion demo, change the serial port and zero offsets, use the open-source hand tracking pipeline, add a screen capture source, and finally write a separate stable tracking program. Therefore, the project was not only a reproduction of an existing design. It became a complete practice in reading, building, testing, changing, and improving an open-source robotic system.

C. Project Objectives

The main goal of this final project was to build and control a working Amazing Hand prototype. More detailed goals were:

- Understand basic robotic hand design ideas through literature review.
- Assemble the Amazing Hand from 3D-printed parts and standard components.
- Test and tune the eight servos through basic Python commands.
- Demonstrate basic hand gestures and motion patterns.
- Run a vision-based hand tracking pipeline using a camera.
- Extend the tracking input so that screen content can also be tracked.
- Write an independent stable tracking program with better response and motion range.
- Record photos and videos to show the assembly process and final functions.

These objectives cover both hardware and software. They also follow the real order of the project. We first learned the field, then built the hand, then made it move,

then made it follow visual hand gestures, and finally improved the tracking program based on the problems observed during testing.

II. Research Report Body

A. Literature Review and Technical Preparation

Our earlier review focused on the development of robotic hands, grasp classification, and several typical hand designs. The main lesson was that robotic hands have no single best design. Different hands are built for different tasks. A prosthetic hand must be light, safe, and easy for a person to wear. A research hand can be heavier, but it may need many degrees of freedom, strong sensors, and precise control. An industrial hand may need high force, high reliability, and simple maintenance.

The review also showed a clear trend from complex imitation toward practical simplification. Early hands such as the Utah/MIT Hand and other high-DOF hands used many actuators and sensors. They were important for research, but they were expensive and difficult to use widely. Later hands used underactuated structures, soft materials, linkage mechanisms, tendon drives, or synergy-based control. These methods reduce the number of motors or reduce the control difficulty. They also make it easier for a hand to adapt to different objects.

The GRASP taxonomy was another useful part of the preparation. It describes human grasp types with dimensions such as opposition type, virtual finger, grasp function, and finger position [2]. This gave us a way to think about hand ability. A robot hand should be evaluated not only by how many joints it has, but also by what useful motions it can perform. In our work, the main evaluation was not a full grasp taxonomy test. Still, the idea helped us choose meaningful demonstrations, such as open hand, close hand, spreading, pointing, victory gesture, and pinch-like motions.

The papers reviewed in our earlier report also compared several recent or typical hands, including Hannes, ILDA, Shadow Dexterous Hand, DEXMART, and a lightweight SMA prosthetic hand [1], [3]–[7]. These designs show the main trade-offs in the field: light weight versus force, simple control versus high dexterity, low cost versus strong sensing, and fast response versus safe motion. These trade-offs also appeared in our much smaller course project. For example, our hand is low-cost and compact, but it has no tactile skin. It can show gestures, but it is not yet a strong object manipulation system.

In addition to literature reading, we studied basic PID control. PID control contains proportional, integral, and derivative terms. The proportional term reacts to present error, the integral term reacts to accumulated error, and the derivative term reacts to the change of error. Even though our final program does not implement a full PID controller, this control view was useful. It helped us understand why a command should not always be sent directly to the motor without filtering. The final hand

tracking program uses smoothing, rate limiting, and lock logic for a similar reason: the input from vision can be noisy, so the output to the servos must be stable.

B. Hardware Platform

The hardware platform was based on the open-source Amazing Hand. The hand has four fingers: index, middle, ring, and thumb. Each finger has two small servos. For the right hand in our project, the servo IDs were arranged as IDs 1–2 for the index finger, IDs 3–4 for the middle finger, IDs 5–6 for the ring finger, and IDs 7–8 for the thumb. The servos were controlled through a serial bus at 1,000,000 baud. The final hardware used a base to hold the hand upright for demonstration.

The Amazing Hand mechanism is different from a simple one-servo-one-joint design. Each finger is driven by a pair of servos through a parallel mechanical structure. In our control model, opposite movement of the two servos is mainly used for curl or flexion-extension. Same-direction movement of the two servos is used to add a splay-like left-right motion. This is not a perfect model of human finger motion, but it is useful for gesture display because it allows each finger to both bend and spread within a limited range.

The hand was built from many 3D-printed parts and several standard parts. The printed parts included finger frames, shells, palm parts, links, and the base. The assembly process required careful fitting because printed plastic parts have tolerances. Small differences in print quality, screw tightness, rod length, and servo horn position can change the final finger posture. For this reason, software offset tuning was a necessary step, not just an optional improvement.

TABLE I
Main Hardware and Software Elements

Element	Description
Mechanical hand	Open-source Amazing Hand right-hand prototype
Fingers	Four fingers: index, middle, ring, thumb
Actuators	Eight Feetech SCS0009 smart servos
Control bus	Serial servo bus, 1,000,000 baud
Basic control	Python with rustypot servo controller
Vision module	MediaPipe Hands with OpenCV camera input
Simulation	MuJoCo-based hand simulation from the demo pipeline
Data pipeline	dora-rs nodes for tracking, simulation, and control
Final script	Independent Python hand tracking and servo mapping program

Table I summarizes the main elements of our system. The system is small compared with advanced robotic hands in research papers, but it is complete enough for learning. It has mechanical structure, actuators, a communication bus, camera sensing, visual processing, simulation, and real-time output to hardware.



Fig. 1. 3D-printed parts before full assembly. The quality and fit of these parts strongly affected later mechanical tuning.

C. Assembly Process

The assembly process started from the printed parts. We first checked the main parts and removed small printing defects. Then we assembled one finger module to understand the mechanical relation between the servo, linkage, and shell. This step was important because a finger module has limited space. If a screw, horn, or rod was placed with a small error, the finger could rub against the shell or move with a strange range.

After testing a single finger, we moved to the hand body. The palm side and back side had to hold the fingers, servos, and wires while still leaving enough room for motion. Wiring was a practical problem. The wires had to be long enough to connect the servos, but not so loose that they



Fig. 2. A single finger module during assembly. Each finger required careful fitting before being installed into the palm.

blocked moving parts. Fig. 3 shows several stages of this process.

One real mechanical problem was a broken printed part. This was not only a small accident. It showed that a 3D-printed robotic hand is sensitive to material strength and assembly force. Some thin structures can break when they are tightened or loaded. We repaired the broken part and also fixed the hand on a base, which made later demonstrations easier and safer. Fig. 4 shows the broken part, repair process, and final prototype.

The final assembly was a working right-hand prototype. It could stand on the base and connect to the control board and computer. At this point, the project changed from mainly mechanical work to mainly software and control work. However, the later software work still depended on the hardware state. If the zero position was wrong, a correct software command could still create a wrong physical posture. For this reason, we tuned the middle position offsets and used safe open and close commands before running more complex tracking programs. During this stage, we wrote and used Amazing-

Hand_ZeroFineTune_All_SAFE.py. This script moved all eight servos through a safe zero-open-close-zero cycle, used a low default speed, inserted command gaps between bus messages, printed every target angle, and tried to return the hand to zero when the user stopped the program. It helped us adjust the MIDDLE_POS_DEG values in a controlled way instead of testing large motions blindly.

D. Basic Servo Control and Gesture Demonstration

The first software layer was a basic Python demo. It used the Scs0009PyController from rustypot to connect to the hand through the serial port. In our local setup, the port was set to COM3. The script defined several movement functions, such as OpenHand, CloseHand, SpreadHand, ClenchHand, Index_Pointing, Victory, Pinched, and Scissors. Each movement was written as target angles for the two servos of each finger.

This stage had two goals. The first goal was to verify that every servo could receive commands and move in the expected direction. The second goal was to build confidence before using camera tracking. Camera tracking can produce changing commands every frame. If the basic open and close positions are not safe, a real-time tracking program can be risky. Therefore, we first tested fixed gestures and simple motion patterns.

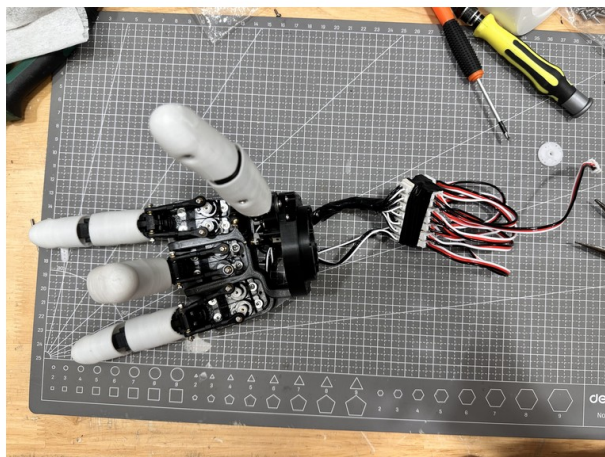
The basic demo also showed the main relation between servo pairs and finger motion. For example, an open hand used angle pairs close to $(-35^\circ, 35^\circ)$, while a closed hand used angle pairs close to $(90^\circ, -90^\circ)$. These values were not pure physical joint angles. They were servo command offsets around a tuned middle pose. Still, they formed a useful low-level control interface.

Video 1 was used to record this stage. It shows that the assembled hand could complete basic gesture motion. This result was important because it proved that the hardware, serial connection, power supply, servo IDs, and basic commands were all usable. It also gave a reference for later vision-based control: the final tracking system should at least be able to produce similar open, close, and pointing-like motions, but with commands generated from hand landmarks rather than fixed functions.

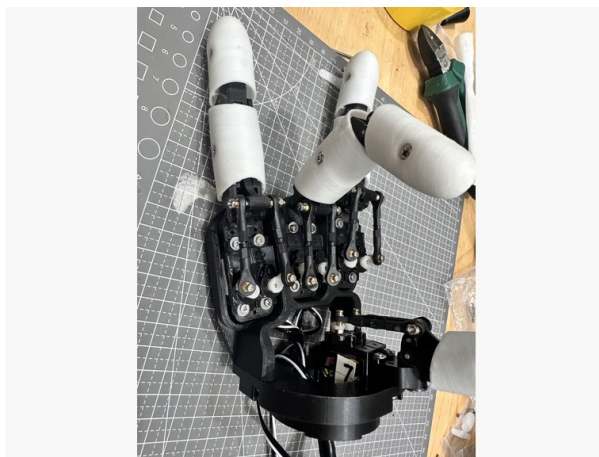
E. Open-Source Hand Tracking Pipeline

After the basic servo demo, we moved to the open-source hand tracking pipeline provided in the Amazing Hand demo directory. This pipeline used three main parts. The first part was a hand tracking node. It used MediaPipe Hands to detect hand landmarks from a camera image [9]. The second part was a simulation node. It used MuJoCo to map tracked hand information to a simulated Amazing Hand posture [10]. The third part was a hardware control node. It sent the simulated joint positions to the real hand through the serial controller.

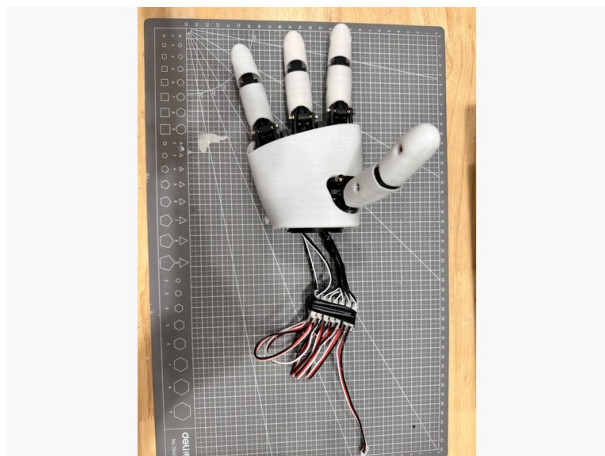
The dataflow file connected these nodes. In our real hardware setup, the hand tracking node sent `r_hand_pos`



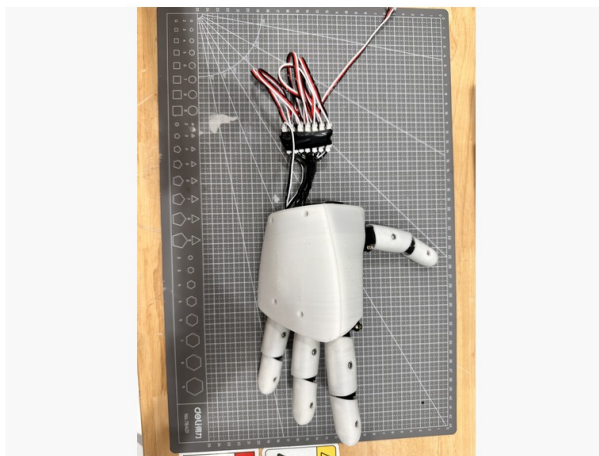
Early assembly



Hand body

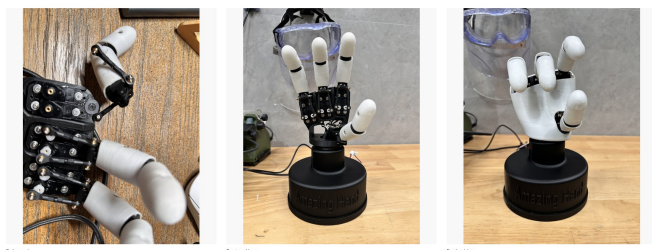


Palm side



Back side

Fig. 3. Main assembly stages: early assembly, hand body, palm side, and back side. The photos show that mechanical assembly and wiring were both important parts of the project.



Broken part

Repair and base

Final prototype

Fig. 4. Mechanical problem and repair: a broken printed part, repaired structure with base, and final assembled prototype.

to the right-hand simulation node. The simulation node produced `mj_r_joints_pos`. The controller node then received these positions and sent commands to the real hand on COM3. The tick rate in the dataflow allowed fast tracking and control updates.

This pipeline was valuable for two reasons. First, it gave

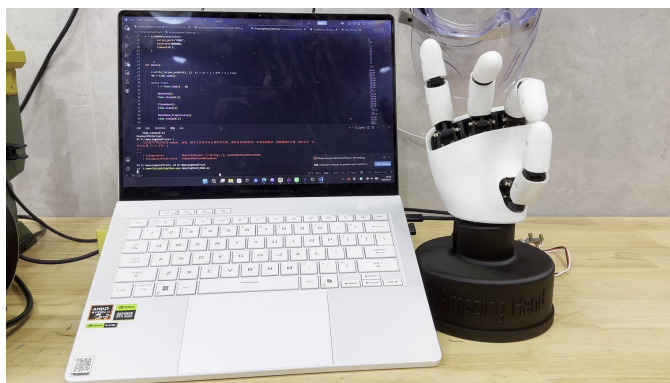


Fig. 5. Representative frame from Video 1. The basic demo script drove the assembled hand through fixed gestures and motion patterns.

us a working example of camera-to-hand control. Second, it used a digital twin. The simulation view made the

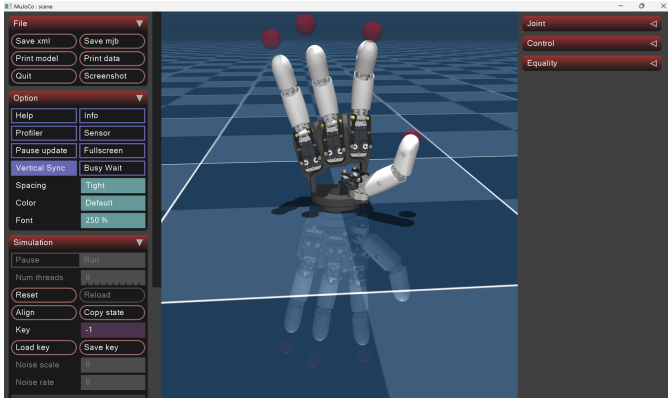


Fig. 6. MuJoCo digital twin used during the tracking pipeline. The digital model helped us observe hand posture before and during real hardware control.

control process easier to understand. Instead of directly watching only the physical hand, we could also watch the simulated hand. This helped separate some problems. If the digital hand moved correctly but the real hand did not, the problem might be in hardware mapping or servo setup. If the digital hand was wrong, the problem might be in tracking or kinematic mapping.

Video 2 was used for this part of the project. It shows camera-based gesture recognition and real hand motion. The tracking result was not perfect, but it proved that the open-source pipeline could connect vision, simulation, and hardware. The hand could follow visible changes in the user’s hand, and the movement was no longer limited to fixed prewritten gesture functions.

F. Screen-Based Hand Tracking Extension

During the project, we also added the ability to track hand gestures from screen content. The original tracking node mainly read frames from a camera. We extended the capture source so that the program could read a selected screen region. This used a screen capture class and allowed the user to choose either camera or screen as the source.

This extension was useful because it made the input more flexible. For example, if a hand video, remote camera window, or another vision window is shown on the screen, the tracking program can process that screen area as if it were a live camera feed. This is not always the best input source, because screen capture adds another layer and may reduce image quality. However, it is useful for testing and demonstration. It also helped us understand the difference between the source of images and the hand landmark algorithm itself. MediaPipe only needs frames. Those frames can come from a camera, a saved video, or a screen region.

In the code, the screen capture class used a rectangle described by left, top, width, and height. It captured that area, converted the frame from BGRA to BGR, and optionally resized it. The rest of the tracking logic did not need to change much. This is a good example of modular

design. The image source was separated from the landmark processing step.

Video 3 records this screen-based tracking result. In the video, the program recognizes a hand gesture displayed on the screen and drives the physical hand. This showed that our pipeline was not limited to a single camera input. It could be adapted to a wider range of visual sources.

G. Independently Written Stable Hand Tracking Program

The final and most important software part was the independent program `Amazing-Hand_HandTrack_FINAL_STABLE.py`. This program did not use the full open-source simulation pipeline. It directly used MediaPipe hand landmarks, calculated finger curl and splay values, mapped them to servo offsets, smoothed the output, and sent commands to the eight servos. It did not include a virtual hand display, but it improved response and motion range for our real hardware demonstration.

The program used OpenCV to read frames from camera index 0. Each frame was flipped for a natural mirror view and passed into MediaPipe Hands. The program tracked one hand with `model_complexity = 0`. This kept the model light enough for real-time use. When a hand was detected, the program drew landmarks on the frame and computed two types of motion command: curl and splay.

1) Finger Curl Estimation: Finger curl was estimated from landmark angles. For each finger, the program took the MCP, PIP, DIP, and tip landmarks. It calculated joint angles such as MCP-PIP-DIP and PIP-DIP-tip. For non-thumb fingers, the final angle was a weighted combination of these two angles. For the thumb, a different weight was used because the thumb has a different structure and visual motion pattern.

The raw curl value was computed from the measured angle:

$$c_{\text{raw}} = \frac{\theta_{\text{open}} - \theta}{\theta_{\text{open}} - \theta_{\text{close}}}, \quad (1)$$

where θ_{open} was set to 165° , and θ_{close} was set to 105° . The result was clamped to the range $[0, 1]$. A value near 0 means open, and a value near 1 means closed. The raw value was then remapped by start, full, and gamma parameters. This made half-bent gestures easier to hold and made the final motion feel more natural.

The thumb used separate parameters. This was necessary because the thumb landmark geometry is different from the other fingers. In our final program, thumb curl started at a smaller raw value and reached full curl earlier. The thumb close command was also slightly amplified. These changes improved the visible thumb motion in the final demonstration.

2) Finger Splay Estimation: The first method for splay would be to use the vector from fingertip to MCP. However, this vector is strongly affected by finger bending.

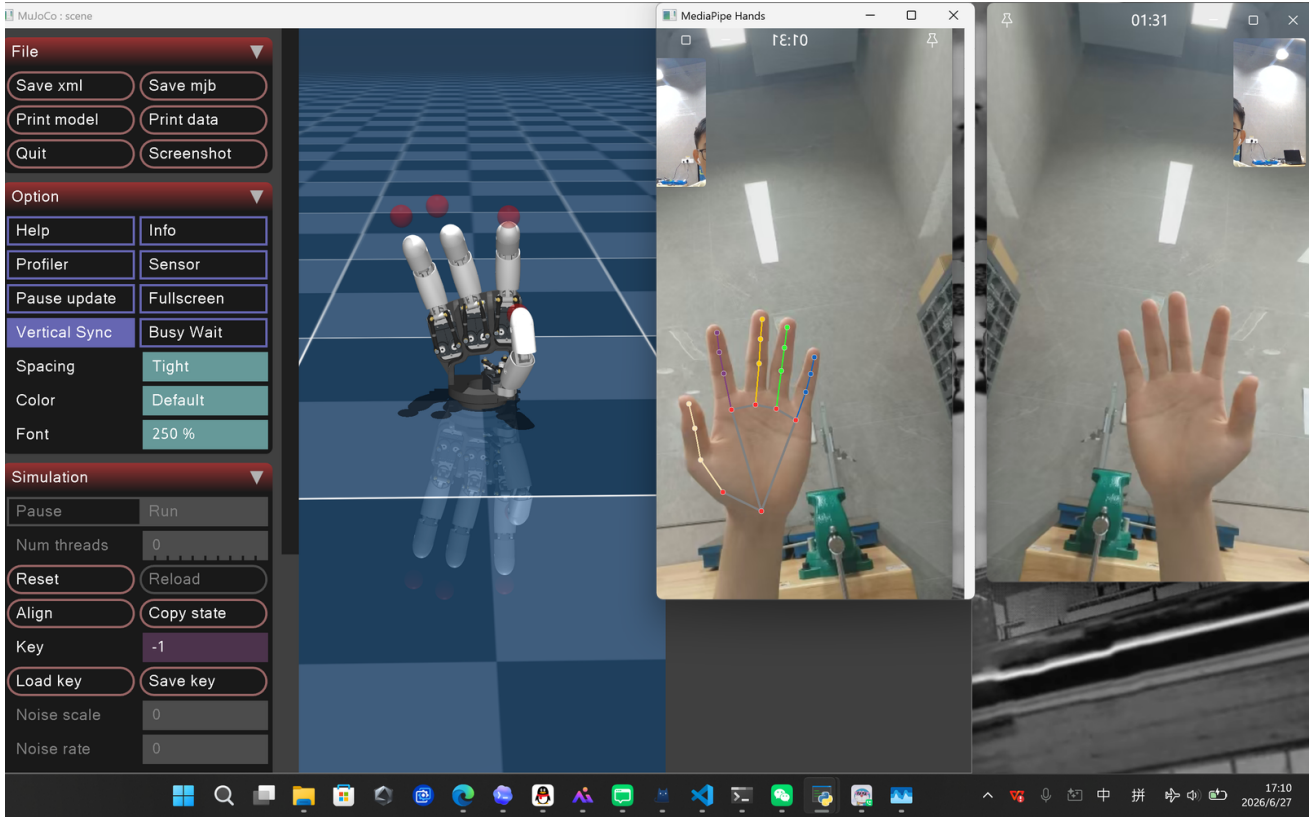


Fig. 7. Tracking and mapping view. The screen shows MediaPipe hand landmarks, camera input, and a simulated hand, which together form a bridge from human gesture to robotic hand motion.



Fig. 8. Representative frame from Video 2. The open-source pipeline used camera input to control the physical hand with the help of simulation.

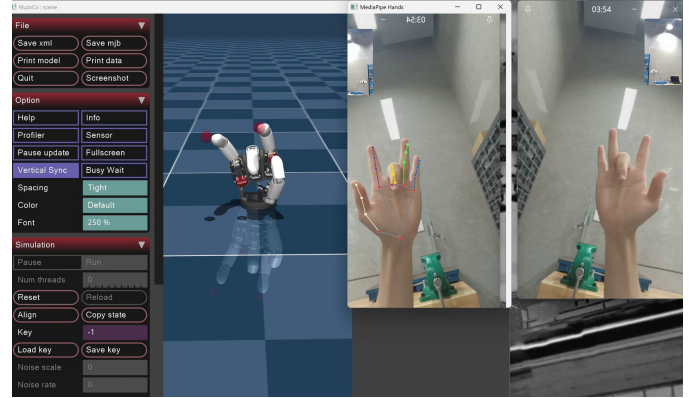


Fig. 9. Representative frame from Video 3. The tracking input came from screen content, and the real hand followed the recognized gesture.

When a finger curls, the fingertip moves a lot, so the program may mistake curl for left-right splay. To reduce this problem, the final script used the MCP-to-PIP direction for the index, middle, and ring fingers. This direction is closer to the proximal phalanx and is less polluted by distal bending.

For the thumb, the program used a mixed measure from CMC-to-MCP and CMC-to-tip. The CMC-to-MCP vector is stable but small, while the CMC-to-tip vector gives a

larger visible range but is more affected by bending. The mixed value gave a practical balance.

The splay value was measured relative to the palm axis. The palm axis was defined by the direction from the index MCP to the pinky MCP. Each finger direction was projected onto this palm axis. The result was a raw splay value for each finger.

3) Neutral Calibration: Human hands and camera views are not always centered. Also, the mechanical hand may have a small left-right bias. For this reason, the program included neutral splay calibration. At the beginning of the program, the user could hold the hand naturally open. The program collected a number of frames and averaged the raw splay values. This average became the neutral splay value.

After calibration, the command used the difference between the present raw splay and the neutral splay:

$$s_i = \text{clip}(k_i \cdot \sigma_i \cdot (r_i - n_i), -1, 1), \quad (2)$$

where r_i is raw splay, n_i is neutral splay, k_i is a gain, and σ_i is a sign term. The result was clipped to $[-1, 1]$. This calibration step made the program easier to use because it did not assume that the user's hand, camera, and mechanical hand all had the same natural center.

4) Temporal Filtering and Curl Lock: MediaPipe tracking can fail when fingers occlude each other. For example, when one finger is bent, part of it may disappear from the camera view. The landmark result may then suddenly show that the finger is opening, even though the user did not open it. If the program directly sends this result to the servos, the mechanical hand may suddenly unfold.

To reduce this problem, the final program added a temporal filter. It follows closing motion quickly but releases opening motion more slowly. In other words, if the new curl is larger than the previous curl, the program can respond fast. If the new curl is smaller, the program limits how much it can drop in one control cycle. This is useful because false opening jumps are more common during occlusion.

The program also added a curl lock. When a finger is judged to be closed above a threshold, it enters a locked state. In this state, the command keeps at least a minimum closed value. The lock is released only after several continuous frames show a clear open value. This prevents a closed finger from slowly opening because of short visual errors.

5) Independent Splay Fade: Another problem was that splay and curl can conflict. When a finger is almost closed, applying a large left-right splay command can make the physical motion look wrong or even pull the finger away from the expected closed gesture. The final program solved this with independent splay fade. The more a finger curls, the less splay is added to that same finger. When the curl is above a cutoff value, the splay for that finger is turned off.

This rule is independent for each finger. It does not use a global fist value. Therefore, if the index finger is extended while the middle and ring fingers are closed, the index finger can still keep left-right splay. This is useful for gestures such as pointing, where one finger is extended and other fingers are closed.

6) Mapping to Servo Offsets: The final step was to map curl and splay into eight servo offsets. The program first set an open offset vector and a close offset vector. For each finger, curl interpolated between the open and close offsets:

$$\mathbf{o}_i = (1 - c_i)\mathbf{o}_{i,\text{open}} + c_i\mathbf{o}_{i,\text{close}}. \quad (3)$$

Then splay was added as a same-direction offset to the two servos of that finger. A trim value was also added to correct a constant bias. Finally, each servo target was clamped by safety angle limits. The program used a control rate of 14 Hz and applied output smoothing before sending commands to the servos. This produced a more stable physical motion.

TABLE II
Key Features of the Final Stable Tracking Program

Feature	Purpose
MCP-PIP splay measure	Reduces confusion between curl and side motion
Thumb-specific splay	Increases visible thumb motion range
Neutral calibration	Removes hand and camera bias
Temporal curl filter	Follows closing fast and opening slowly
Curl lock	Prevents closed fingers from opening due to occlusion
Independent splay fade	Keeps extended fingers free while closed fingers stay stable
Angle limits	Protects servos and printed parts from unsafe commands
Output smoothing	Reduces sudden jumps in real hardware motion

Table II lists the main improvements. These changes were simple, but they were based on real problems observed during testing. This is an important part of engineering work. The final result was not produced by one perfect formula. It was produced by repeated testing, watching the mechanical hand, finding the cause of wrong motion, and adding limited fixes that matched the problem.



Fig. 10. Representative frame from Video 4. The final independent tracking program controlled the physical hand with improved response and larger visible motion.

Video 4 records the final optimized version. Compared with the earlier tracking pipeline, this version focused on direct hardware response. It did not show a virtual hand, but the physical hand motion was more useful for our final demonstration. The hand reacted to camera gestures with better speed and larger motion range, especially after tuning thumb motion, splay gain, curl filtering, and lock thresholds.

H. Results and Evaluation

The project produced several concrete results. First, we completed a real Amazing Hand prototype from printed parts and standard components. The final hand could stand on a base and connect to the computer through the control board. Second, the basic Python script could drive all four fingers through fixed gestures. Third, the open-source tracking pipeline could use a camera, MediaPipe, simulation, and hardware control to make the hand follow gestures. Fourth, the screen-based tracking extension could process gestures from screen content. Fifth, the final independent script could control the physical hand directly with better practical behavior.

The four videos were used as the main visual evidence. Video 1 shows basic gesture motion. Video 2 shows camera-based hand tracking with the open-source pipeline. Video 3 shows screen-based tracking. Video 4 shows the final independently written stable tracking program. These videos are not just extra media. They show the project path from basic motion to final tracking control.

The main success of the project is integration. The work connected many different parts: literature review, mechanical assembly, servo calibration, serial communication, Python control, MediaPipe landmarks, simulation, screen capture, and real-time hardware output. Each part alone is not enough. The course value came from making all parts work together on a real device.

The final program also improved the practical control quality. Using MCP-to-PIP direction for splay reduced the error caused by finger bending. Neutral calibration made the system easier to start. Temporal filtering and curl lock reduced false opening caused by occlusion. Independent splay fade made gestures with both extended and closed fingers more stable. These improvements were especially important because the hand had no tactile feedback and the camera did not always see every finger clearly.

However, the project also has clear limits. The first limit is sensing. Our system mainly uses vision. It has no tactile skin, force sensing, or object contact feedback. Therefore, it can show gestures, but it cannot yet grasp unknown objects safely. The second limit is visual reliability. MediaPipe performs well in many cases, but it can still fail under occlusion, bad lighting, fast motion, or unusual hand poses. The third limit is mechanical precision. Printed parts, screw tightness, and servo offsets all affect the final posture. The fourth limit is evaluation. We mainly used qualitative tests and videos. We did not perform a full

quantitative study of response time, angle error, grasp force, or long-term durability.

Even with these limits, the project met the course goals. It changed a set of open-source files and printed parts into a real working prototype. It also gave us a direct experience of the gap between a software idea and a physical robot. In simulation or code, a command is only a number. On the real hand, the same command can cause delay, friction, noise, weak motion, or mechanical stress. This experience is difficult to get from reading papers only.

III. Conclusion

This course project built a complete Amazing Hand system and used it for gesture control. The work started with paper reading and basic control learning. The literature review helped us understand the main trends in robotic hand design, including simplified actuation, soft or flexible structures, grasp taxonomy, and trade-offs between dexterity, weight, cost, and control difficulty. The PID study helped us think about response, stability, and filtering.

On the hardware side, we assembled a four-finger, eight-servo Amazing Hand prototype. We handled printed parts, finger modules, palm and back shells, wiring, base fixing, and a broken part repair. We then tuned basic servo offsets with our safe zero fine-tuning script and tested fixed gestures through a Python demo. This proved that the hardware and communication path worked.

On the software side, we first used the open-source hand tracking pipeline with MediaPipe, dora-rs, MuJoCo simulation, and real hardware control. We then extended the tracking input to support screen capture. Finally, we wrote an independent stable tracking program. This final program directly maps hand landmarks to servo targets and includes neutral calibration, curl filtering, curl lock, independent splay fade, angle limits, and smoothing. It improved the visible response and motion range of the real hand.

The project shows that an open-source robotic hand can be a strong learning platform. It is low-cost enough for a course project, but complex enough to expose real engineering problems. Future work should add more quantitative tests, such as response time, angle accuracy, and repeatability. It should also add tactile or force feedback so the hand can move from gesture display toward safer object grasping. A cleaner user interface and better automatic calibration would also make the system easier to use.

Acknowledgment

We thank the course instructor and teaching staff for guidance during the project. We also thank the developers of the Amazing Hand open-source project for providing the mechanical design, assembly documents, and example code. We thank the authors of MediaPipe Hands, MuJoCo, and dora-rs for tools that made the vision and simulation

parts possible. Finally, we thank all group members for their work in paper reading, assembly, debugging, filming, and report preparation.

Appendix A

Photo Materials Used in the Report

The report used selected photos from the project record folder. The photos were chosen to cover both the assembly process and the final result. The printed parts and single-finger photos show the start of the mechanical work. The wiring, palm, and back-side photos show the middle assembly process. The broken-part and repair photos show a real engineering problem. The final prototype photo shows the completed hand.

TABLE III
Main Photo Evidence

Figure File in Report	Meaning
assembly_printed_parts.jpg	Printed parts before assembly
assembly_single_finger.jpg	One finger module during assembly
assembly_process_collage.jpg	Several stages of hand assembly
assembly_issue_repair_final.jpg	Broken part, repair, and final prototype
digital_twin.png	MuJoCo digital hand model
tracking_mapping.png	Mapping between tracking and simulation

Appendix B

Video Materials Used in the Report

All four project videos were used. Because a printed PDF cannot play video directly, each video is represented by a frame in the main text and described in Table IV. The full video files are kept in the original project material folder. They are also available through the [USTC cloud share link](#).

TABLE IV
Video Evidence

Video	Report Frame	Content
Video 1	Fig. 5	Basic fixed gesture motion from the Python demo
Video 2	Fig. 8	Camera hand tracking with the open-source pipeline
Video 3	Fig. 9	Screen-based gesture tracking and real hand output
Video 4	Fig. 10	Final stable tracking program written for this project

Appendix C

Main Code Files

The project used and modified several code files. The basic gesture demo was based on `AmazingHand_Demo.py`. It controlled the eight servos through fixed motion functions. The middle offset tuning used our self-written `AmazingHand_ZeroFineTune_All_SAFE.py`; this script safely cycled the hand between zero, open, close, and zero positions

while printing the target angle of each servo. The open-source tracking pipeline used `dataflow_tracking_real.yml`, which connected the hand tracker, simulation node, and hardware controller. The hand tracking node was based on `HandTracking/main.py`; it used MediaPipe Hands and OpenCV. The final direct-control version was `AmazingHand_HandTrack_FINAL_STABLE.py`. This file contained the main independent contribution of the software part.

TABLE V
Selected Parameters in the Final Stable Program

Parameter	Value or Role
SERIAL_PORT	COM3
BAUDRATE	1,000,000
SERVO_IDS	1 to 8
CONTROL_HZ	14 Hz command update rate
SMOOTHING_ALPHA	0.25 output smoothing factor
AUTO_CALIBRATE_NEUTRAL	Automatic neutral splay calibration
CURL_LOCK_ENABLE	Prevents false opening after finger closure
SPLAY_FADE_WITH_CURL	Reduces splay when a finger is bent

References

- [1] M. Laffranchi et al., “The Hannes hand prosthesis replicates the key biological properties of the human hand,” *Science Robotics*, vol. 5, no. 46, Sep. 2020, doi: 10.1126/scirobotics.abb0467.
- [2] T. Feix, J. Romero, H.-B. Schmedmayer, A. M. Dollar, and D. Kragic, “The GRASP taxonomy of human grasp types,” *IEEE Transactions on Human-Machine Systems*, vol. 46, no. 1, pp. 66–77, Feb. 2016.
- [3] C. Piazza, G. Grioli, M. G. Catalano, and A. Bicchi, “A century of robotic hands,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 2, pp. 1–32, 2019, doi: 10.1146/annurev-control-060117-105003.
- [4] U. Kim et al., “Integrated linkage-driven dexterous anthropomorphic robotic hand,” *Nature Communications*, vol. 12, p. 7177, 2021, doi: 10.1038/s41467-021-27261-0.
- [5] H. Yang et al., “A lightweight prosthetic hand with 19-DOF dexterity and human-level functions,” *Nature Communications*, Jan. 2025, doi: 10.1038/s41467-025-56352-5.
- [6] Shadow Robot Company, “Shadow Dexterous Hand technical specification.” [Online]. Available: <https://www.shadowrobot.com/products/dexterous-hand/>
- [7] G. Palli et al., “The DEXMART hand: Mechatronic design and experimental evaluation of synergy-based control for human-like grasping,” *The International Journal of Robotics Research*, vol. 33, no. 5, pp. 799–824, 2014.
- [8] Pollen Robotics, “Amazing Hand project.” [Online]. Available: <https://github.com/pollen-robotics/AmazingHand>
- [9] F. Zhang et al., “MediaPipe Hands: On-device real-time hand tracking,” arXiv:2006.10214, 2020.
- [10] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.
- [11] dora-rs, “dora-rs documentation.” [Online]. Available: <https://dora-rs.ai/>